HealthBuddy AI Medical Chatbot

Medical Chatbot – Implementation Explanation

1. Data Ingestion from PDFs

The first step is collecting the medical knowledge that the chatbot will use to answer questions.

- Source: Medical book PDFs such as *Gale Encyclopedia of Medicine* with `637 pages`.
- These PDFs are stored in a **data/** folder in the project.

Using **LangChain**, the PDFs are loaded using:

- `PyPDFLoader`
- `DirectoryLoader`

These tools read every page of the PDF and convert them into **Document objects** containing text.

Goal:
Convert raw PDF files into machine-readable text so that they can be processed by the chatbot.

---

2. Text Preprocessing and Chunking

Large language models like **GPT-4** cannot process extremely large texts at once because they have a **limited context window (~8192 tokens)**.

Therefore, the text must be divided into smaller pieces.

Using **RecursiveCharacterTextSplitter** from **LangChain**:

- `chunk_size = 500`
- `chunk_overlap = 20`

Why Overlap?

Overlap ensures that important information is not lost between chunks.

Example:

Chunk 1 → contains part of a medical definition
Chunk 2 → overlaps slightly so the meaning stays connected.

After chunking, the large PDF becomes **thousands of smaller text chunks (~5800 chunks)**.

Purpose:
Make the data easier for the model to process while preserving the meaning.

---

3. Creating Embeddings

The text chunks are then converted into **vector embeddings**.

Embedding model used:

sentence-transformers/all-MiniLM-L6-v2 from **Hugging Face**

What Embeddings Do

Embeddings convert text into **numerical vectors** that represent semantic meaning.

Example:

```
"headache symptoms"
→ [0.123, -0.542, 0.876, ...]
```

Each chunk becomes a **384-dimensional vector**.

Why this is important:

Embeddings allow the system to perform **semantic search**, meaning it can find information based on meaning instead of exact words.

---

4. Storing Data in Vector Database

All embeddings are stored in a vector database called **Pinecone**.

Setup Steps

1. Create a Pinecone account.
2. Generate an API key.
3. Create a vector index.

Configuration:

- Index Name: `medical-chatbot`
- Dimension: **384**
- Similarity Metric: **Cosine Similarity**

Using LangChain's vector store integration:

```
PineconeVectorStore.from_documents()
```

This uploads all document embeddings to Pinecone.

Each stored vector contains:

- embedding vector

- metadata (source, page number)

Purpose:
Allow fast retrieval of relevant medical information when a user asks a question.

---

5. Retriever + LLM (RAG Architecture)

The chatbot uses **Retrieval-Augmented Generation (RAG)**.

Step 1: Retriever

The retriever searches Pinecone for the **most relevant text chunks**.

Configuration:

```
search_type = "similarity"
k = 3
```

This means it retrieves the **top 3 most relevant chunks**.

Step 2: LLM

The retrieved context is sent to the language model:

GPT-4o

Using **ChatOpenAI** from **LangChain**.

The prompt template instructs the model to:

- answer based only on the retrieved medical content
- provide clear and concise responses
- avoid hallucinations

Step 3: RAG Chain

The pipeline works like this:

```
User Question
      ↓
Retriever searches Pinecone
      ↓
Top 3 relevant medical chunks
      ↓
Chunks + User Question sent to GPT-4o
```

```
      ↓
Final Answer Generated
```

Result:
The chatbot gives **domain-specific medical answers based on the book data**.

---

6. Flask Web Application

To allow users to interact with the chatbot, a web application is created using **Flask**.

Frontend

Simple chat interface built with:

- HTML
- CSS

Files:

```
templates/chat.html
static/style.css
```

This page contains:

- chat box
- message input
- send button
- response display

Backend

The Flask application handles user queries.

Routes used:

Home Route

```
/
```

Loads the chat interface.

Chat API

```
/chat
```

Workflow:

1. User sends a question.
2. Flask sends the query to the RAG chain.
3. Retriever fetches relevant medical chunks.
4. GPT-4 generates the answer.
5. Response is sent back to the frontend.

---

7. Running the Application Locally

To test the chatbot locally:

```
python app.py
```

The server runs on:

```
http://localhost:8080
```

Users can open the browser and interact with the chatbot.

---

8. Deployment

The project is deployed on **Hugging Face Spaces.

This allows the chatbot to be accessible online without running it locally.

Users can:

- open the web interface
- ask medical questions
- receive answers generated using the medical knowledge base.

✅ **Final Result**

The system works as a **Medical Knowledge Chatbot** that:

- Reads medical books
- Converts them into embeddings
- Stores them in Pinecone
- Retrieves relevant knowledge
- Uses GPT-4 to generate accurate answers
- Provides responses through a web chat interface.